
Elicitation Matters: How Prompts and Query Protocols Shape LLM Surrogates under Sparse Observations

Ge Lei

Dyson School of Design Engineering
Imperial College London
Exhibition Road, London SW7 2AZ
g.lei23@imperial.ac.uk

Samuel J. Cooper

Dyson School of Design Engineering
Imperial College London
Exhibition Road, London SW7 2AZ
samuel.cooper@imperial.ac.uk

Abstract

Large language models are increasingly used as surrogate models for low-data optimization, but their optimizer-facing prediction and its uncertainty remain poorly understood. We study the surrogate belief elicited from an LLM under sparse observations, showing that it depends strongly on prompt text and query protocol. We introduce an uncertainty-alignment criterion that measures whether model uncertainty tracks residual ambiguity among sample-consistent functions. Across controlled inference tasks and Bayesian optimization studies, we find that structural prompts act as effective priors, POINTWISE and JOINT querying induce different beliefs, and sequential evidence leads to non-monotonic, order-sensitive confidence updates. These effects change downstream acquisition decisions and regret, showing that elicitation protocol is part of the LLM surrogate specification, not a formatting detail.

1 Introduction

Large language models can perform nontrivial in-context learning (ICL) without gradient updates [2, 4, 1]. This has motivated interest in using them as surrogate regressors inside experiment design and optimization workflows [7, 13, 3]. Most evaluations of such systems focus on predictive accuracy and, increasingly, calibration against realized error [5, 14]. Those are useful diagnostics, but they do not fully capture the regime that matters most for early-stage optimization.

In the low-data setting, the objective is typically underdetermined: a small observation set constrains many plausible completions. In this regime, the optimizer needs more than a point estimate. It needs a notion of uncertainty that captures the remaining degrees of freedom in objectives still consistent with the observations. It should also update this uncertainty sensibly when new evidence is introduced. Classical surrogates make these commitments explicit through function classes, priors, and posterior updates [21, 19]. LLM surrogates instead often inherit implicit regularities from pretraining, prompt wording, and query protocol.

For LLM surrogates, the optimizer-facing belief is elicited through language. However, linguistic competence is not the same as surrogate behavior. An LLM may correctly explain descriptors such as *monotonic*, *quadratic*, or *underdetermined*, but that does not imply that these descriptors constrain the numerical predictions or uncertainty estimates it returns. What matters for optimization is the operational belief induced by the full elicitation procedure.

Prior work has mainly evaluated LLMs by their numerical prediction accuracy or by their performance inside Bayesian optimization loops. We instead ask what surrogate the optimizer actually receives from an LLM under sparse observations. Given the same observations D , prompt text p and query

protocol π can change the predictions, uncertainty estimates, and induced predictive distributions seen by the optimizer. We call this optimizer-facing object a *protocol-conditioned surrogate belief*. Here, *belief* is operational: it denotes observable predictive behavior, not the model’s internal state or an approximate Bayesian posterior. Our goal is not to show that prompts can change LLM outputs, but to characterize the surrogate belief induced by the full elicitation procedure: how prompt text and query protocol shape predictions, uncertainty, induced predictive distributions, downstream optimization decisions, and sequential updates under new evidence. We make four contributions:

- We propose an *uncertainty-alignment* criterion that compares model-side uncertainty with residual sample-consistent ambiguity.
- We show that prompt language acts as an effective prior: correct structural cues help, while incorrect cues systematically mislead the surrogate.
- We show that POINTWISE and JOINT querying induce distinct surrogate beliefs: POINTWISE is more ambiguity-sensitive and observation-faithful, while JOINT is more coherent but less constraint-faithful.
- We show that under sequential evidence, LLM surrogate beliefs exhibit a Dunning–Kruger-like dip-and-recovery in confidence, with evidence order shifting the confidence valley.

2 Problem Setting

We study sparse-observation function inference: an LLM observes a small set of (x, y) pairs from an unknown objective and predicts values at unseen points. Our interest is the predictive object exposed to a downstream optimizer, and whether its uncertainty tracks the ambiguity left by sparse data.

2.1 Sparse-observation function inference

Let $f^* : \mathcal{X} \rightarrow \mathbb{R}$ denote the unknown objective. After t evaluations, the observation set is

$$D_t = \{(x_i, y_i)\}_{i=1}^t, \quad y_i = f^*(x_i). \quad (1)$$

For a reference class $\mathcal{F}$, define the sample-consistent set

$$\mathcal{H}(D_t; \mathcal{F}) = \{f \in \mathcal{F} : f(x_i) = y_i \ \forall (x_i, y_i) \in D_t\}. \quad (2)$$

In our controlled tasks, $\mathcal{H}(D_t; \mathcal{F})$ remains nontrivial, so point accuracy and ambiguity-awareness need not coincide.

2.2 Protocol-conditioned surrogate belief

Fix a query set $\mathbf{x} = (x_1, \dots, x_m)$ with m queried locations, and a prompt specification (p, π) , where p is prompt text and π is query protocol. The elicited predictive law over the corresponding value vector $\mathbf{y} \in \mathbb{R}^m$ is

$$B_\theta^{p, \pi}(D_t; \mathbf{x}) := P_\theta(\mathbf{y} \mid \mathbf{x}, D_t, p, \pi). \quad (3)$$

Definition 2.1 (Protocol-conditioned surrogate belief). For fixed (D_t, p, π) , the family

$$\{B_\theta^{p, \pi}(D_t; \mathbf{x}) : m \geq 1, \mathbf{x} \in \mathcal{X}^m\}$$

is the *protocol-conditioned surrogate belief* exposed by the LLM. Here “belief” is behavioral: it is the predictive object seen by a downstream optimizer, not a claim about internal latent state.

Under POINTWISE, queries are answered independently:

$$B_\theta^{p, \text{POINTWISE}}(D_t; \mathbf{x}) = \prod_{j=1}^m P_\theta(y_j \mid x_j, D_t, p, \text{POINTWISE}). \quad (4)$$

Under JOINT, the full query list is answered in one autoregressive completion:

$$B_\theta^{p, \text{JOINT}}(D_t; \mathbf{x}) = P_\theta(\mathbf{y} \mid \mathbf{x}, D_t, p, \text{JOINT}) = \prod_{j=1}^m P_\theta(y_j \mid y_{<j}, \mathbf{x}, D_t, p, \text{JOINT}). \quad (5)$$

Definition 2.2 (Belief shift at fixed evidence). For two prompt specifications (p, π) and (p', π') , and any divergence D between distributions on $\mathbb{R}^m$, define

$$\Delta_D(\mathbf{x}; D_t, (p, \pi), (p', \pi')) = D\left(B_\theta^{p, \pi}(D_t; \mathbf{x}), B_\theta^{p', \pi'}(D_t; \mathbf{x})\right). \quad (6)$$

A nonzero value means that the elicited surrogate changes although the observations do not.

Remark 2.1 (Effective prior, operationally). We use *effective prior* to mean systematic belief shift at fixed D_t ; this is a behavioral rather than Bayesian claim.

For $Y = (Y_1, \dots, Y_m) \sim B_\theta^{p, \pi}(D_t; \mathbf{x})$ and any linear statistic $S_w = \sum_{j=1}^m w_j Y_j$ with weights $w_j \in \mathbb{R}$,

$$\text{Var}(S_w) = \sum_{j=1}^m w_j^2 \text{Var}(Y_j) + 2 \sum_{j < k} w_j w_k \text{Cov}(Y_j, Y_k). \quad (7)$$

Thus two protocols can agree on pointwise uncertainty yet differ on curve-level uncertainty through cross-location dependence.

2.3 Prompt axes and study design

We vary three prompt dimensions: **query protocol** (POINTWISE, which answers each query independently, or JOINT, which answers all queried locations in a single autoregressive completion), **prompt style** (a neutral baseline, an UNDERDETERMINED warning that states that the observations do not identify a unique function, or an unrelated warning control), and **structural information** (ranging from no objective information to increasingly specific function family, structural, and parameter cues, as well as intentionally incorrect information). Representative templates are given in Appendix A.

The function families used in our experiments—Gaussian, quadratic, linear, sinusoidal, and logistic—are chosen for interpretability and control, rather than as a uniquely privileged benchmark. Similarly, the model suite is designed to span diverse sizes and families, including Llama, GPT, and Qwen variants, covering open- and closed-weight as well as uni- and multimodal models (see Appendix B for the full list). Importantly, our elicited beliefs target intuitive (System 1–like) judgments rather than deliberative (System 2–like) reasoning, without eliciting chain-of-thought.

2.4 Model-side uncertainty proxies

Token-level uncertainty. For a greedy numeric prediction $\hat{y}(x)$ with output tokens $s_{1:L}$, we use

$$\text{NLL}_\theta^{\text{tok}}(\hat{y}(x)) = -\frac{1}{L} \sum_{\ell=1}^L \log P_\theta(s_\ell \mid s_{<\ell}, x, D_t, p, \pi), \quad (8)$$

with confidence proxy $c_\theta^{\text{tok}}(x) = \exp(-\text{NLL}_\theta^{\text{tok}}(\hat{y}(x)))$.

Sampling-based uncertainty. For output-level uncertainty, we repeatedly sample under fixed (x, D_t, p, π) at temperature = 1 and define

$$u_\theta^{\text{sam}}(x) = \text{Var}\left(\hat{y}^{(1)}(x), \dots, \hat{y}^{(n)}(x)\right). \quad (9)$$

2.5 Reference ambiguity and uncertainty alignment

Realized error alone is insufficient under underdetermination. We therefore introduce a reference ambiguity profile

$$v_{\mathcal{H}}(x; D_t) = \text{Var}_{f \sim q_D}[f(x)], \quad (10)$$

where q_D is a reference distribution over functions that remain sample-consistent with D_t .

For a controlled parametric family f_ϕ with parameter $\phi \in \Phi$, we define

$$q_D(\phi) \propto p_0(\phi) \mathbf{1}\left[\max_{(x_i, y_i) \in D_t} |f_\phi(x_i) - y_i| \leq \varepsilon_{\text{cons}}\right], \quad (11)$$

where p_0 is a broad proposal distribution and $\varepsilon_{\text{cons}}$ is the consistency tolerance. We approximate $v_{\mathcal{H}}$ by Monte Carlo on a fixed query grid (Appendix D); it is a reference signal for residual ambiguity, not a claim about true posterior variance.

Definition 2.3 (Uncertainty alignment). For a query grid $\mathbf{x} = (x_1, \dots, x_m)$ and a pointwise model-side uncertainty proxy $u(x)$, define

$$A(u; D_t, \mathbf{x}) = \rho((u(x_j))_{j=1}^m, (v_{\mathcal{H}}(x_j; D_t))_{j=1}^m), \quad (12)$$

where ρ denotes Spearman rank correlation.

We report $A(u_{\theta}^{\text{sam}}; D_t, \mathbf{x})$ and the analogous token-level version using $\text{NLL}_{\theta}(\hat{y}(x))$. This asks whether more ambiguous regions are assigned higher uncertainty.

A useful idealized baseline is the width functional

$$W_{\mathcal{H}}(x; D_t) = \sup_{f, g \in \mathcal{H}(D_t; \mathcal{F})} |f(x) - g(x)|. \quad (13)$$

Proposition 2.1 (Consistency-set contraction). *If $D_t \subset D_{t+1}$, then $\mathcal{H}(D_{t+1}; \mathcal{F}) \subseteq \mathcal{H}(D_t; \mathcal{F})$, and hence*

$$W_{\mathcal{H}}(x; D_{t+1}) \leq W_{\mathcal{H}}(x; D_t) \quad \forall x \in \mathcal{X}. \quad (14)$$

Remark 2.2 (Sequential implication). Exact sample-consistent ambiguity cannot expand as new observations are added. Any temporary rise in model uncertainty or drop in confidence under sequential evidence therefore reflects model-side belief revision, not feasible-set expansion alone.

3 Static Belief under Underdetermination

Unlike analytic surrogates, where priors must be built into kernels, architectures, or features, LLM surrogates can receive soft structural information in natural language. We ask whether this verbal structure changes optimizer-facing predictions, rather than merely eliciting textual acknowledgment.

3.1 Prompt structure changes the implied prior

Holding D_t fixed and varying only the structural language in p isolates a prompt-induced belief shift in the sense of Definition 2.2. Under POINTWISE querying, Figure 1A shows that, with sparse observations, correct structural descriptions pull predictions toward the intended family, whereas wrong-type descriptions push them away. Prompt language therefore acts as an effective prior over completions in the sense of Remark 2.1. The figure also reveals that independently elicited predictions surprisingly remain coherent across query locations. We further discuss this tendency in Section 3.2.

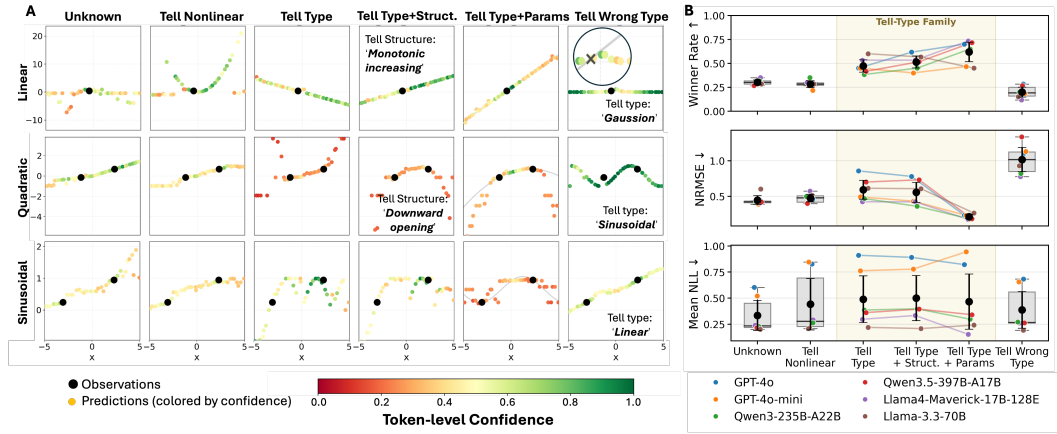


Figure 1: **A:** GPT-4o pointwise completions. **B:** Aggregate prompt effects across models.

We summarize the elicited curves across models and 4 function families, using two metrics: *winner rate*, the fraction assigned to the correct function family by AICc-based family selection (Appendix C.1), and NRMSE, which measures pointwise fit (Appendix C.2). Figure 1B shows that Tell-Type Family prompts improve family recovery and NRMSE relative to Unknown, whereas

Tell Wrong type degrades both. Adding more correct structural information further improves recovery within the same family. Prompt changes also affect uncertainty, as reflected by shifts in mean NLL. Tell Nonlinear provides only a weak cue and has little effect on fit, but can still change uncertainty. Changes in uncertainty are also model-dependent: larger models such as GPT-4o and Qwen tend to become more confident as more structural information is provided, whereas some lower-capacity models such as GPT-4o-mini and Llama-3.3-70B become more diffuse under more complex formula descriptions.

Additionally, even without observations, LLM beliefs can show structured, model-dependent trends such as linearity or monotonicity (Appendix A). These trends can persist far outside the main range $[-5, 5]$, surprisingly even for inputs as large as 10^1 – 10^4 (Appendix B). However, LLMs are not always able to verbalize these numerical biases (Appendix C). Predicting a single point can often be solved through direct completion, whereas describing the global shape requires integrating a higher-level abstraction. This highlights a key gap: verbal reasoning is not equivalent to the internal computation underlying numerical prediction.

Together, these results support two conclusions: (1) the elicited LLM surrogate is shaped by both sparse numerical observations and prompt language; (2) structural prompts act as effective priors, with correct cues improving fit and wrong-type cues misleading the surrogate.

3.2 POINTWISE and JOINT are not interchangeable elicitation protocols

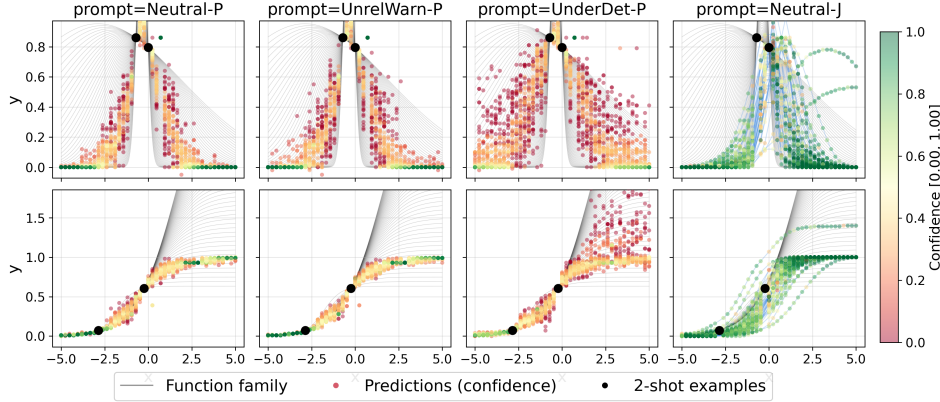


Figure 2: 50 repeated completions of the same sparse-observation task in GPT-4o. Under POINTWISE, UNDERDETERMINED broadens plausible completions while preserving coarse structure.

We next hold D_t fixed and change only the elicitation protocol. The task uses two observations and a partially specified hypothesis class in which the underlying family is Gaussian or logistic. We sample 50 untruncated temperature-1 completions per condition to compare POINTWISE and JOINT with minimal decoding confounds.

In Figure 2, point color denotes token-level uncertainty and the background shows the sample-consistent set $\mathcal{H}(D_t; \mathcal{F})$. Under POINTWISE, independently answered locations still organize into coherent curves, revealing a strong default bias (which would otherwise be scattered without such bias). Adding an UNDERDETERMINED warning broadens the prediction distribution in more ambiguous regions. Under JOINT, later values condition on earlier generated values, so completions become more path-dependent and internally coherent, but less faithful to the original observations. Interestingly, beyond curve-level consistency, we observe additional biases in the Gaussian setting: predictions rarely exceed an apparent upper bound near 1, and completions often treat the two observed points as lying on opposite sides of a symmetry axis rather than on the same tail.

Figure 3 quantifies these differences across six models. Panel A estimates belief shift at fixed evidence via Optimal Transport (OT) distance between sampled completion distributions. Under POINTWISE, UNDERDETERMINED moves substantially away from neutral while the unrelated-warning control remains close; under JOINT, these distances shrink, suggesting that it compresses prompt effects. Panel B reports the pointwise Fit Recoverability Score (FRS; Appendix C.2.2), which measures whether repeated predictions can be explained by one consistent fit from the stated family. Panel C

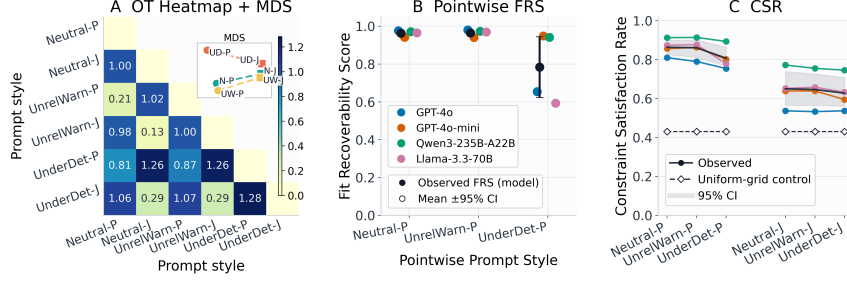


Figure 3: Aggregate effects of prompt style and protocol. **A:** OT distance (with Multidimensional Scaling inset). **B:** fit recoverability score. **C:** constraint satisfaction rate.

reports the Constraint Satisfaction Rate (CSR; Appendix C.2.1), which measures whether predictions are consistent with the stated function family. Under POINTWISE UNDERDETERMINED, FRS is generally lower, consistent with reduced inductive bias. Under JOINT, CSR is lower, consistent with autoregressive error propagation. This also explains why token-level and sampling-based uncertainty can diverge: JOINT outputs may be locally confident yet vary across runs.

These results support three conclusions: (1) query protocol is part of the LLM surrogate specification, not a formatting detail: JOINT suppresses prompt effects and weakens constraint fidelity, whereas POINTWISE more faithfully exposes prompt-induced variation; (2) neutral POINTWISE prompting reveals a bias toward globally coherent completion; (3) task-relevant underdetermination warnings broaden POINTWISE completion distribution while largely preserving compatibility with the stated family, although effect sizes vary by model.

4 Uncertainty Alignment

Standard calibration asks whether confidence tracks realized error. Under underdetermination, this is incomplete: a prediction may be accurate yet overconfident because many alternative functions remain consistent with the observations. Definition 2.3 instead asks whether the model’s uncertainty correctly reflects which locations are inherently more ambiguous. We compute the reference ambiguity profile in Eq. (10) using the Monte Carlo sample-consistent construction in Section 2.5 and Appendix D, and compare it with LLM-side uncertainty via Spearman correlation. We use both token level (NLL) and sampling variance as LLM uncertainty proxies. Because the diagnostic is rank-based, these proxies need not share a scale; the question is simply whether more ambiguous regions receive higher uncertainty.

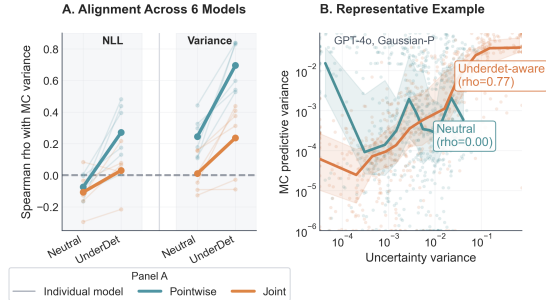


Figure 4: Uncertainty alignment. **A:** Spearman correlation. **B:** Gaussian example (GPT-4o).

Figure 4 shows the main pattern. Neutral prompts often give weak or negative alignment. In POINTWISE mode, UNDERDETERMINED-aware prompting improves both variance- and NLL-based alignment: uncertainty increases in ambiguous regions and decreases in less ambiguous ones. The same intervention is much weaker in JOINT mode. In both query protocols, the LLM understands the underdetermination warning at a semantic level, but this does not mean it can incorporate it into an optimizer-facing surrogate belief in both cases.

5 Belief Updating under Sequential Evidence

Previous sections study LLM surrogate belief under a fixed observation set. Since optimization is sequential, the surrogate should also update as evidence accumulates. We therefore examine how the induced surrogate evolves as evidence is sequentially accumulated under different ordering conditions.

For each ordering o , observations form nested prefixes $D_1^{(o)} \subset \dots \subset D_T^{(o)}$. Non-conflict orders reveal the same candidate pool by decreasing information (High Info), increasing information (Low Info), or random permutation, where information is quantified by GP-estimated posterior-variance reduction over the query grid. These conditions therefore isolate pure order effects under identical evidence. The conflict orders insert one contradictory point at a midway or late prefix step in the same high-information support sequence, chosen at a GP-confident location and shifted away from the GP prediction (Appendix F). After each prefix, we record accuracy $1 - \text{NRMSE}_t$ and mean token confidence $\bar{c}_t$. We summarize the lowest-confidence point by the *confidence valley* step $t_{\text{valley}} = \arg \min_t \bar{c}_t$ (Appendix C). For comparison with a conventional surrogate, we replay the same prefixes with a fixed-kernel GP surrogate and map uncertainty to confidence by $1 - \sigma_t^{\text{GP}}(x)$.

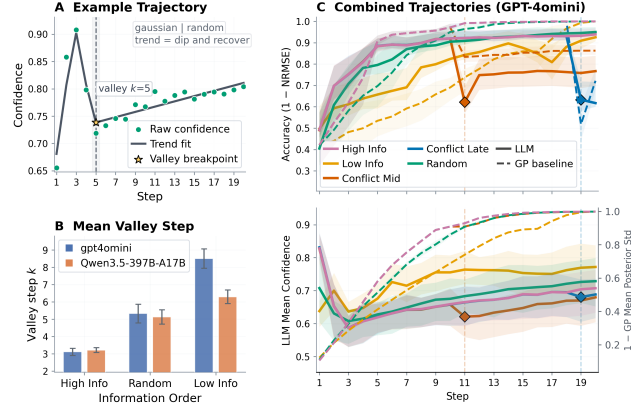


Figure 5: Sequential belief updating. A: Example run. B: Valley step. C: Accuracy/confidence trajectories.

Figure 5A shows a dip-and-recovery confidence trajectory rather than monotonic contraction. As an informal analogy, this resembles the Dunning–Kruger effect in psychology [9]: individuals may initially overestimate their ability, but confidence drops as expertise increases, producing a *valley of despair*¹. As expertise continues to accumulate, confidence then recovers, yielding the observed dip-and-recovery pattern. Figure 5B shows that the order of evidence shapes when the confidence valley occurs. High Info, where GP-ranked high-information observations are revealed first, moves the valley earlier, consistent with faster revision of the initial hypothesis. Low Info delays the valley, leaving the model confident but weakly updated for longer. Conflict points cause sharper disruption: accuracy drops abruptly and confidence destabilizes, with Conflict Late recovering least because fewer supportive observations remain.

The GP control tracks evidence geometry more directly: uncertainty changes monotonically, contracting earlier under *high_info_first*, more gradually under *low_info_first*, and being locally perturbed by the inserted conflict point. The stronger non-monotonicity in the LLM therefore reflects model-side belief revision rather than evidence geometry alone. Unlike the GP, which reaches the same final state for a fixed observation set, the LLM remains order-sensitive and can produce different outcomes under different evidence orders.

Overall, the LLM surrogate updates as more evidence is provided, but not solely as a function of the final observation set. Since exact consistency would only reduce residual ambiguity as observations accumulate, the non-monotonic confidence curves in Figure 5 indicate internal belief revision rather than simple feasible-set narrowing.

6 Do Elicited Belief Differences Affect BO Decisions?

Previous sections reveal differences in elicited surrogate beliefs, but not whether they affect optimization behavior. We therefore use the LLM predictions as BO surrogates and measure their effect. These experiments are not general-purpose optimizer benchmarks; instead, they test whether belief differences persist under BO decisions. Acquisition scores are computed from numeric LLM predictions and token-level uncertainty: UCB-style ranking score for Branin and an EI-style ranking score for HPO and battery design. The uncertainty is used as a heuristic exploration signal, not as a calibrated objective-space predictive standard deviation. The three studies are designed to cover increasingly realistic settings: a controlled benchmark with deliberate prior mismatch, a practical HPO task with limited domain structure, and a battery-design task where the prompt can provide useful

¹The analogy is descriptive only. We use it to name the shape of the confidence trajectory, not to claim that the LLM has human-like metacognition or a psychological self-assessment process.

scientific context. All BO studies use 10 seeds per condition, and all LLM calls use direct numeric prediction without chain-of-thought. The LLMs are GPT-4o for Branin, Qwen3.5-397B-A17B for HPO, and GPT-5.4 for battery design. These studies are not intended as cross-model comparisons. For each task, we use a representative LLM and compare prompt/protocol variants within the same task. Additional experimental details and prompts are provided in Appendix G.

Controlled prior-mismatch benchmark. We first run a small BO loop on canonical Branin and on a modified Branin variant. In both cases, the objective is described only as an unknown function. The modified variant preserves the Branin-like shape but moves the optima away from the canonical locations; the position of the initial observations is also chosen to have values similar to canonical Branin. This creates a setting in which a familiar Branin prior can be useful on the canonical task but misleading when the optimum is displaced.

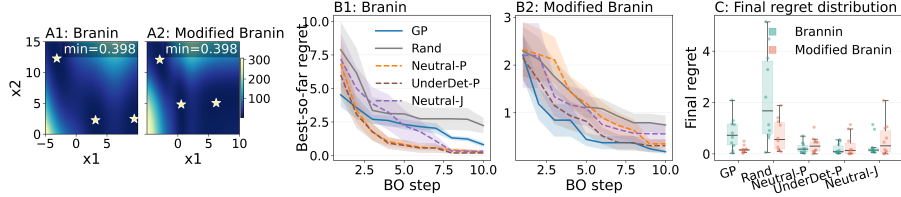


Figure 6: Branin BO study. A: Objectives. B: Mean best-so-far regret. C: Final regret.

Figure 6 shows the effect of prior mismatch. On canonical Branin, LLM POINTWISE achieve lower regret than the GP baseline, but this advantage disappears once the optima are modified, consistent with the fact that the LLM prior influences BO decisions. POINTWISE outperforms JOINT, and the underdetermination-aware prompt further helps on the modified task, matching the static diagnostics: POINTWISE better fits the observations, while the underdetermination prompt improves alignment. Thus, the prompt and protocol effects observed statically also lead to different BO decisions.

HPO benchmark. We next test whether these protocol effects persist in a more practical setting: low-budget XGBoost hyperparameter optimization on California Housing. The LLM receives model family and task type, but not the dataset name or any dataset-specific semantic description. Figure 7 shows that POINTWISE reduces regret fastest in early stages, while JOINT is weaker for much of the trajectory. SMAC3 achieves the best final regret. These results suggest that the POINTWISE–JOINT difference is not limited to toy functions, and that LLM priors can drive rapid early-stage progress even when only weak task structure is provided.

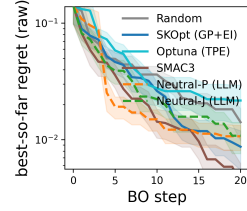


Figure 7: Mean best-so-far regret (HPO).

Domain-structured battery design. Finally, we evaluate a domain-structured cathode-design task: optimizing positive-electrode thickness and porosity using FDN-simulated energy densities at 0.5C and 3C. The objective rewards high energy density at both rates while penalizing imbalance. Unlike black-box baselines, the LLM receives indirect domain information through variable names, battery context, and the objective definition; encoding this equivalently in classical optimizers would require task-specific surrogate design and is not always well-defined. The warning prompt further emphasizes broad domain guidance (the LLM may already possess): low-rate energy density improves with lower inactive content, while high-rate performance depends more strongly on transport pathways and porosity. But it does not reveal the simulator response surface or good designs. Figure 8 shows the largest benefit for LLM surrogates. Both POINTWISE prompts achieve the best optimization performance, rapidly identifying high-scoring regions. The warning prompt further improves performance, suggesting that informative domain guidance affects downstream optimization decisions.

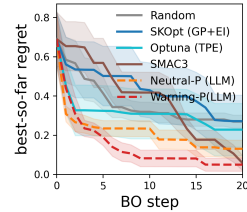


Figure 8: Mean best-so-far regret (battery).

Across the BO studies, we use optimization as a test of elicited beliefs rather than as a broad optimizer comparison. The Branin study shows that LLM priors can help under alignment but hurt under mismatch. The HPO study shows that the POINTWISE–JOINT gap persists beyond synthetic objectives. The battery task shows that LLM surrogates are most useful when the prompt exposes domain structure that generic black-box baselines cannot access. Together, these results show

that prompt and protocol affect BO decisions, and should be treated as part of the LLM surrogate specification. LLMs also make BO more accessible through language-specified priors.

7 Related Work

In-context learning and surrogate regression. A growing literature views in-context learning as inference-time adaptation [2, 4]. Closely related, Requeima et al. [17] introduce LLM Processes, which elicit numerical predictive distributions from LLMs conditioned on natural language. Our work is complementary: we study the optimizer-facing surrogate belief elicited from an LLM, focusing on how this belief depends on observations, prompt text, and query protocol. Beyond predictions, we analyze uncertainty under static evidence and sequential evidence updates.

LLMs for Bayesian optimization. LLMs have also been used to improve Bayesian optimization [13, 19, 18]. These works primarily ask whether LLMs can improve optimization performance. By contrast, we hold the broad optimization setup fixed and ask how prompt text and query protocol affect the surrogate belief used by the acquisition function.

Mechanistic views of in-context learning. A parallel literature studies the mechanisms underlying ICL. Some interpret it as implicit Bayesian inference over latent task structure [22], while others show that transformers can implement simple learning algorithms, such as linear regression or gradient-based updates, in the forward pass [4, 1, 20]. Mechanistic interpretability work further links ICL to circuit-level components such as induction heads [15]. We treat LLMs as distinct from these mechanisms, even if they may exhibit similar behaviors. We do not adjudicate among these mechanisms. Our use of “belief” is behavioral: it refers to the finite-dimensional predictive law elicited from a fixed LLM.

Uncertainty and calibration. Calibration [6, 11, 16] has recently been studied for LLM uncertainty [8, 10]. These works typically evaluate uncertainty against realized error. In sparse-observation, however, accurate predictions can still be overconfident when many functions remain consistent with the data. We therefore study *uncertainty alignment*: whether LLM uncertainty tracks residual function-space ambiguity. Related work finds that multi-turn LLM confidence need not improve monotonically as evidence accumulates [12]. We further show that the resulting confidence valley is order-sensitive, with its location depending on the sequence in which evidence is revealed.

8 Discussion and Conclusions

This paper studies how prompt text and query protocol affect the behavior of an LLM-based surrogate under sparse observations. The central object is the *protocol-conditioned surrogate belief*: the optimizer-facing predictive law induced by observations, prompt text, and query protocol. It treats uncertainty alignment, structural recovery, constraint fidelity, sequential revision, and BO regret as different projections of the same elicited object and asks whether prompt language actually changes the surrogate seen by the optimizer rather than merely being verbally reproduced. Our controlled experiments show that LLM surrogates are shaped not only by data, but also by how they are elicited. Correct structural language acts as an effective prior, while wrong-type language can systematically mislead the surrogate. POINTWISE and JOINT are not interchangeable interfaces: POINTWISE better preserves ambiguity-sensitive uncertainty and observation faithfulness, whereas JOINT yields coherent but less constraint-faithful completions. Under sequential evidence, the surrogate revises in non-monotonic and order-sensitive ways. In BO, these differences affect acquisition behavior and regret. In LLM-based BO, elicitation protocol is therefore not an implementation detail; it changes the finite-dimensional predictive law used by the acquisition function.

Broader Impact. In real low-data optimization settings such as robotic-lab control, decisions must integrate sparse observations with safety limits, resource constraints, machine precision, and human expectations about plausible values. The practical question is how much the elicited surrogate should rely on LLM prior knowledge and observed evidence. Prompt design and query protocol are therefore part of the LLM surrogate specification because they shape the balance between prior-driven completion, observation faithfulness, and global coherence in downstream decisions. **Limitations.** The study is intentionally controlled: the tasks are mostly noise-free, and the ambiguity reference is defined only for families where a sample-consistent profile can be approximated. This improves interpretability, but leaves open how the same effects scale to noisier settings.

References

- [1] Ekin Akyürek, Dale Schuurmans, Jacob Andreas, Tengyu Ma, and Denny Zhou. What learning algorithm is in-context learning? investigations with linear models. *arXiv preprint arXiv:2211.15661*, 2022.
- [2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] Chih-Yu Chang, Milad Azvar, Chinedum Okwudire, and Raed Al Kontar. Llinbo: Trustworthy llm-in-the-loop bayesian optimization. *arXiv preprint arXiv:2505.14756*, 2025.
- [4] Shivam Garg, Dimitris Tsipras, Percy S Liang, and Gregory Valiant. What can transformers learn in-context? a case study of simple function classes. *Advances in neural information processing systems*, 35: 30583–30598, 2022.
- [5] Jiahui Geng, Fengyu Cai, Yuxia Wang, Heinz Koepl, Preslav Nakov, and Iryna Gurevych. A survey of confidence estimation and calibration in large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 6577–6595, 2024.
- [6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q Weinberger. On calibration of modern neural networks. In *International conference on machine learning*, pages 1321–1330. PMLR, 2017.
- [7] Hao Hao, Xiaoqun Zhang, and Aimin Zhou. Large language models as surrogate models in evolutionary algorithms: A preliminary study. *Swarm and Evolutionary Computation*, 91:101741, 2024.
- [8] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 2021.
- [9] Justin Kruger and David Dunning. Unskilled and unaware of it: how difficulties in recognizing one’s own incompetence lead to inflated self-assessments. *Journal of personality and social psychology*, 77(6):1121, 1999.
- [10] Lorenz Kuhn, Yarin Gal, and Sebastian Farquhar. Semantic uncertainty: Linguistic invariances for uncertainty estimation in natural language generation. *arXiv preprint arXiv:2302.09664*, 2023.
- [11] Volodymyr Kuleshov, Nathan Fenner, and Stefano Ermon. Accurate uncertainties for deep learning using calibrated regression. In *International conference on machine learning*, pages 2796–2804. PMLR, 2018.
- [12] Chengzu Li, Han Zhou, Goran Glavaš, Anna Korhonen, and Ivan Vulić. Can large language models achieve calibration with in-context learning? In *ICLR 2024 Workshop on Reliable and Responsible Foundation Models*, 2024.
- [13] Tennison Liu, Nicolás Astorga, Nabeel Seedat, and Mihaela van der Schaar. Large language models to enhance bayesian optimization. *arXiv preprint arXiv:2402.03921*, 2024.
- [14] Xiaoou Liu, Tiejun Chen, Longchao Da, Chacha Chen, Zhen Lin, and Hua Wei. Uncertainty quantification and confidence calibration in large language models: A survey. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V. 2*, pages 6107–6117, 2025.
- [15] Catherine Olsson, Nelson Elhage, Neel Nanda, Nicholas Joseph, Nova DasSarma, Tom Henighan, Ben Mann, Amanda Askell, Yuntao Bai, Anna Chen, et al. In-context learning and induction heads. *arXiv preprint arXiv:2209.11895*, 2022.
- [16] Yaniv Ovadia, Emily Fertig, Jie Ren, Zachary Nado, David Sculley, Sebastian Nowozin, Joshua Dillon, Balaji Lakshminarayanan, and Jasper Snoek. Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. *Advances in neural information processing systems*, 32, 2019.
- [17] James Requeima, John Bronskill, Dami Choi, Richard E Turner, and David Duvenaud. Llm processes: Numerical predictive distributions conditioned on natural language. *Advances in Neural Information Processing Systems*, 37:109609–109671, 2024.
- [18] Bobak Shahriari, Kevin Swersky, Ziyu Wang, Ryan P Adams, and Nando De Freitas. Taking the human out of the loop: A review of bayesian optimization. *Proceedings of the IEEE*, 104(1):148–175, 2015.
- [19] Jasper Snoek, Hugo Larochelle, and Ryan P Adams. Practical bayesian optimization of machine learning algorithms. *Advances in neural information processing systems*, 25, 2012.

- [20] Johannes Von Oswald, Eyvind Niklasson, Ettore Randazzo, João Sacramento, Alexander Mordvintsev, Andrey Zhmoginov, and Max Vladymyrov. Transformers learn in-context by gradient descent. In *International Conference on Machine Learning*, pages 35151–35174. PMLR, 2023.
- [21] CK Williams and CE Rasmussen. Gaussian processes for machine learning, mit press. *Cambridge, MA*, (2 (3)), 2006.
- [22] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. An explanation of in-context learning as implicit bayesian inference. *arXiv preprint arXiv:2111.02080*, 2021.

A Prompt Templates and Experimental Inputs

This appendix documents the experimental inputs used throughout the paper. The purpose of these prompt manipulations is diagnostic: they are designed to isolate which aspects of prompt text and query protocol change the optimizer-facing surrogate belief, i.e., the finite-dimensional predictive law defined in the main text.

A.1 Base task framing

We use two base prompt formats corresponding to the two query protocols studied in the paper.

Base prompt: pointwise (POINTWISE). In the pointwise setting, each query location is asked in a separate completion. A representative template is:

```
You are a function approximator. Given examples of  $(x, y)$  pairs from a function, predict
the  $y$  value for a new  $x$ .
Rules:
1. Output only the numeric  $y$  value.
2. Use the same precision as the examples.
3. Do not include explanation or extra text.
Here are example points from an unknown function:
[1]  $x = x_1$   $y = y_1$ 
[2]  $x = x_2$   $y = y_2$ 
...
Predict the  $y$  value for:
 $x = x^{\text{query}}$ 
 $y =$ 
```

Base prompt: joint (JOINT). In the joint setting, the full query list is answered in a single completion. A representative template is:

```
You are a function approximator. Return strict JSON only. Output must contain exactly one
key "y" with a list of numeric values. The list order must match the query list order.
Here are example points from an unknown function:
[1]  $x = -1.00$   $y = 0.20$ 
[2]  $x = 0.00$   $y = 1.00$ 
[3]  $x = 1.00$   $y = 0.25$ 
Predict  $y$  for each query in order.
QUERY LIST:
[1]  $x = -0.50$ 
[2]  $x = 0.50$ 
[3]  $x = 1.50$ 
[4]  $x = -1.50$ 
Return: {"y": [y1, ..., y4]}
```

The pointwise format is repeated independently for each query point on the evaluation grid. The joint format produces one vector-valued completion per repeated sample.

A.2 Prompt style conditions

We study three prompt styles.

Neutral baseline. The neutral condition presents the task directly, with no additional statement about ambiguity.

Underdetermination-aware warning. This condition makes the ambiguity of the sparse observation set explicit. A representative sentence is:

```
Warning: this description is underdetermined. With only a few points, many functions of the
stated family can still fit the observations.
```


Its purpose is not to instruct the model toward any specific function, but to test whether making residual ambiguity explicit changes the predictive distribution or uncertainty.

Unrelated warning control. This control preserves the presence of an extra warning-like sentence while removing its task relevance. A representative sentence is:

Warning: descriptive details such as color, texture, and smell can matter when discussing food.

This distinguishes task-relevant ambiguity effects from generic instruction-loading effects.

A.3 Structural-information conditions

We vary the amount and correctness of verbal structural information supplied in the prompt, while keeping the observed input-output examples fixed. The prompt styles are:

Unknown. The model is explicitly told that the function type is unknown, and must infer the pattern only from the examples:

IMPORTANT: The underlying function type is unknown.

Tell non-linear. The model receives only a coarse shape-level cue, without the function family or parameters:

IMPORTANT: The underlying function is nonlinear, not a straight line.

Tell type. The model is told the true function family and its generic formula family, but not the instance-specific parameters:

IMPORTANT: The underlying function is {type_desc}.
Formula family: {formula}.

Tell type + structure. The model is told the true function family, its generic formula family, and one qualitative structural hint derived from the sampled instance:

IMPORTANT: The underlying function is {type_desc}.
Formula family: {formula}.
{structural_hint}

Tell type + parameters. The model is told the true function family, the generic formula family, and the numerical parameters of the sampled instance:

IMPORTANT: Here are example points from a {type_desc} function.
The exact formula is {formula}. Parameters: {params_str}.

Wrong type. The model is intentionally given an incorrect function-family label:

IMPORTANT: The underlying function is {wrong_type}.

A.4 Function families and task instances

The paper uses several small controlled function families rather than one monolithic benchmark suite.

- The static-structure study uses one-dimensional analytic families chosen so that sparse observations leave multiple plausible completions; representative rows in Figure 1 include linear, quadratic, and sinusoidal examples.
- The protocol and uncertainty-alignment studies emphasize Gaussian and logistic families because they provide clear underdetermined regions between and beyond sparse observations.
- The sequential-updating study uses one-dimensional tasks with observation orders that can be manipulated by informativeness or conflict.

- The downstream optimization study uses the two-dimensional Branin function and a shifted Branin variant.

The common design principle is that the observation set is intentionally sparse enough for prompt text and protocol to matter.

A.5 Output processing and repeated sampling

For POINTWISE, each completion is parsed as a single numeric prediction. For JOINT, each completion is parsed as a numeric list whose length must match the query list length. Throughout the paper, outputs are requested at the same displayed precision as the examples. This keeps formatting variance from masquerading as predictive variance.

Repeated querying is used for two purposes: to estimate sampling-based dispersion and to construct empirical completion distributions. In the protocol-comparison experiments we use 50 repeated samples per condition, which is the setting used in the figures in the main text. When a completion cannot be parsed into the required numeric format, it is treated as invalid for metrics that require numeric predictions; the same parsing rule is applied uniformly across conditions.

B Models in the Figure

Table B.1 lists the models used in the paper.

Table B.1: Models shown in the figure.

Legend Name	Full Model Name
GPT-4o	GPT-4o
GPT-4o-mini	GPT-4o mini
GPT-5.4	GPT-5.4
Qwen3-235B-A22B	Qwen3-235B-A22B
Llama-3.3-70B	Llama 3.3 70B Instruct
Qwen3.5-397B-A17B	Qwen3.5-397B-A17B
Llama4-Maverick-17B-128E	Llama 4 Maverick (17Bx128E)

C Metric Definitions

This section defines the metrics used in the main paper. Each metric is a functional of either a single elicited completion curve or an empirical sample from the protocol-conditioned surrogate belief. The purpose of this section is therefore not merely bookkeeping: it clarifies which projection of the elicited surrogate each metric measures.

C.1 Shape identification and winner rate

To quantify whether a sampled completion recovers the correct *shape family*, we fit a small set of candidate families to the predicted curve and select the best one by corrected Akaike information criterion (AICc).

Let

$$\mathcal{C} = \{\text{gaussian}, \text{sinusoidal}, \text{quadratic}, \text{linear}\}.$$

For a trial t , the predicted family is

$$\widehat{\text{family}}_t = \arg \min_{c \in \mathcal{C}} \text{AIC}_{t,c}. \quad (15)$$

For candidate family c ,

$$\text{AIC}_{t,c} = n \log \left(\frac{\text{RSS}_{t,c}}{n} + 10^{-12} \right) + 2K_c, \quad (16)$$

$$\text{AIC}_{c,t,c} = \text{AIC}_{t,c} + \frac{2K_c(K_c + 1)}{\max(n - K_c - 1, 1)}, \quad (17)$$

where n is the number of queried points, $\text{RSS}_{t,c}$ is the residual sum of squares for family c , and $K_c = k_c + 1$ is the total parameter count, including the residual variance parameter. The mean-function parameter count is

$$k_c = \begin{cases} 2, & \text{linear,} \\ 3, & \text{quadratic,} \\ 4, & \text{gaussian or sinusoidal.} \end{cases}$$

For a group of trials G , the winner rate is

$$\text{WinnerRate}(G) = \frac{1}{|G|} \sum_{t \in G} \mathbf{1}[\widehat{\text{family}}_t = \text{family}_t^*]. \quad (18)$$

Winner rate is a shape-level metric: it rewards recovery of the correct family even when pointwise error is not minimal. In the terminology of the main paper, it is a structural-identification functional on sampled draws from the elicited surrogate belief.

C.2 NRMSE

For each trial t , pointwise fit quality is measured by normalized root mean squared error

$$\text{NRMSE}_t = \frac{\sqrt{\frac{1}{m} \sum_{j=1}^m (\hat{y}_t(x_j) - f_t^*(x_j))^2}}{y_t^{\max} - y_t^{\min} + \varepsilon}, \quad (19)$$

where $\{x_j\}_{j=1}^m$ is the evaluation grid and $y_t^{\max}, y_t^{\min}$ are the maximum and minimum ground-truth values on that grid.

C.2.1 Constraint Satisfaction Rate (CSR)

CSR measures whether a predicted query value remains compatible with the stated function family and the two observed anchors. For each case c , let (x_{1c}, y_{1c}) and (x_{2c}, y_{2c}) denote the two anchor points, and let $(x_{cj}, \hat{y}_{cj})$ denote a predicted query point. We define

$$\text{CSR}_{f,g} = \frac{1}{\sum_{c \in \mathcal{C}_g} n_c} \sum_{c \in \mathcal{C}_g} \sum_{j=1}^{n_c} \mathbf{1}[\mathcal{F}_f((x_{1c}, y_{1c}), (x_{2c}, y_{2c}), (x_{cj}, \hat{y}_{cj})) = 1], \quad (20)$$

where $\mathcal{F}_f$ is a family-specific feasibility predicate implemented by analytic or closed-form checks with numerical tolerances. Thus CSR is not computed by fitting the predicted point to the ground-truth curve, nor by measuring prediction error. It asks whether the predicted point could belong to some member of the stated family that is also compatible with the observed anchors.

For the linear family, feasibility reduces to the tolerance check

$$\left| \hat{y}_{cj} - \left(y_{1c} + \frac{y_{2c} - y_{1c}}{x_{2c} - x_{1c}} (x_{cj} - x_{1c}) \right) \right| \leq \varepsilon_{\text{lin}},$$

with $\varepsilon_{\text{lin}} = 10^{-3}$ in our implementation.

For the Gaussian family, feasibility is checked by an analytic log-shape test. When the three candidate values are positive and the three x locations are distinct, we compute the unique quadratic interpolant through $(x_{1c}, \log y_{1c})$, $(x_{2c}, \log y_{2c})$, and $(x_{cj}, \log \hat{y}_{cj})$. The triple is considered Gaussian-feasible only if the resulting quadratic has negative curvature, up to numerical tolerance $\varepsilon_{\text{gauss}} = 10^{-12}$.

For the logistic family, feasibility is checked by a three-point analytic existence test for the no-offset logistic form. The three values must be positive and have distinct x locations; we then test whether there exists an upper asymptote L larger than all three values such that the logit-transformed points

$$\log \frac{y}{L - y}$$

are collinear in x , up to tolerance $\varepsilon_{\text{logistic}} = 10^{-3}$.

For the quadratic family, any three finite observations at distinct x locations are feasible because a quadratic interpolant exists. If two candidate points share the same x location, feasibility requires their y values to agree up to the same numerical tolerance used for repeated-location checks. For the exponential family with offset, three non-conflicting observations are treated analogously as feasible under the three-parameter form $A \exp(rx) + b$.

In Figure 3C, the relevant comparison is between the left group (POINTWISE) and the right group (JOINT). Lower CSR under JOINT indicates that autoregressive completion more easily weakens adherence to the observed anchors. Within the pointwise group, CSR remains comparatively high across prompt styles, which is why we interpret warning-induced diversity in POINTWISE as structured rather than arbitrary. Operationally, CSR is an anchor-faithfulness functional: it asks whether a predicted point still belongs to some completion compatible with the stated family and the observed anchors.

C.2.2 Pointwise Fit Recoverability Score (FRS)

FRS measures whether the predicted outputs for one case can still be well explained by a single fitted curve within the target family. For each case c , we first fit a curve within family f and compute

$$\text{MSE}_c = \frac{1}{n_c} \sum_{j=1}^{n_c} \left(\hat{y}_{cj} - h_f(x_{cj}; \hat{\theta}_c) \right)^2, \quad s_c = \exp\left(-\frac{\text{MSE}_c}{\tau_f}\right), \quad \text{FRS}_{f,g} = \frac{1}{|C_g|} \sum_{c \in C_g} s_c.$$

Here τ_f is a family-specific scale parameter that keeps the score comparable across cases.

Panel B in Figure 3 should be read within the pointwise setting only. Lower FRS means that repeated pointwise completions are less tightly concentrated around a single recoverable family fit. In the main text, we therefore use a reduction in FRS as evidence that task-relevant warnings broaden the candidate completion distribution and make repeated outputs less stable. Operationally, FRS is a one-curve recoverability functional on repeated samples from the elicited belief.

C.3 Sequential-updating metrics

For a trial and evidence order o , let $\bar{c}_t$ denote mean token-level confidence over the fixed query grid after step t . We report:

- stepwise accuracy $\text{Acc}_t = 1 - \text{NRMSE}_t$;
- mean confidence $\bar{c}_t$;
- confidence-valley timing $t_{\text{valley}} = \arg \min_t \bar{c}_t$;
- confidence-valley depth $\Delta_{\text{valley}} = \bar{c}_1 - \min_t \bar{c}_t$.

The bar plot in Figure 5 summarizes the empirical distribution of t_{valley} across trials. The term “confidence valley” is purely descriptive: it denotes the step of minimum mean model confidence under a reveal order, not a claim about human cognition.

C.4 Bayesian-optimization metrics

At BO step t , after selecting one or more new points and updating the dataset, we record the incumbent value

$$y_t^{\text{best}} = \max_{(x_i, y_i) \in D_t} y_i \quad (21)$$

in the maximization convention used in the paper. Simple regret is then

$$r_t = f^*(x^*) - y_t^{\text{best}}, \quad (22)$$

where x^* is the global maximizer of the benchmark objective on the search domain. Final regret is r_T at the end of the BO horizon. In addition to regret curves and final-regret distributions, Figure 6 reports representative spatial query traces and within-method uncertainty-versus-error scatter plots as diagnostics of search behavior.

A Model-Dependent Belief Patterns

Without observations, LLM beliefs may still form structured patterns rather than scattered points. These implicit patterns vary across models, as shown in Figures A.1 and A.2.

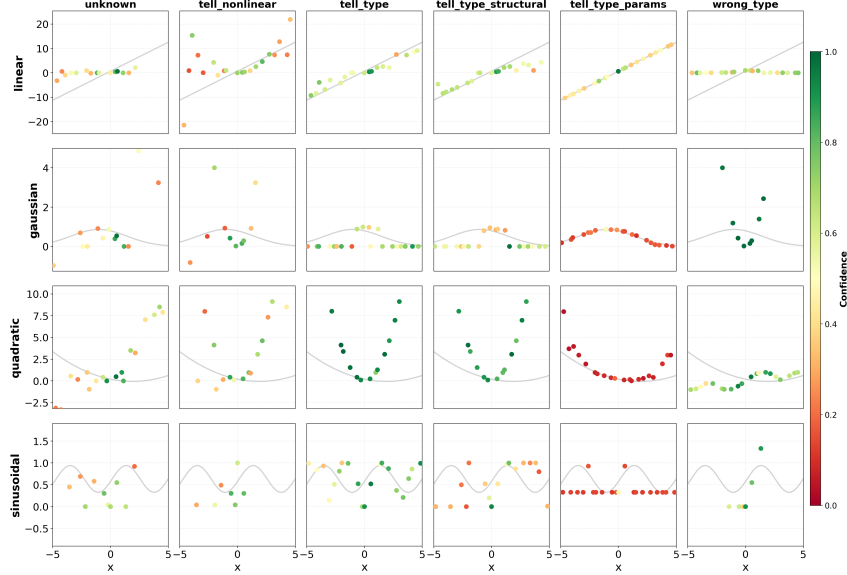


Figure A.1: Belief patterns of GPT-4o without observations.

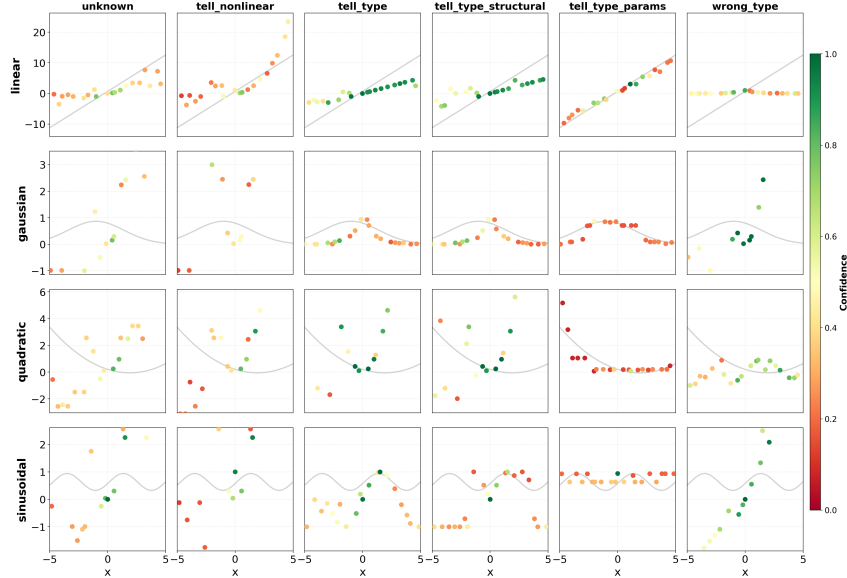


Figure A.2: Belief patterns of GPT-4o-mini without observations.

B Extrapolation Beyond the Familiar Numerical Range

In the main prompt-effect experiments, query inputs are sampled from the interval $[-5, 5]$. This range contains small numerical values that are likely to be common in the pretraining distribution of large language models. To examine whether the observed behavior is specific to this familiar numerical

range, we ran an additional diagnostic experiment in which the model was asked to extrapolate to much larger input values.

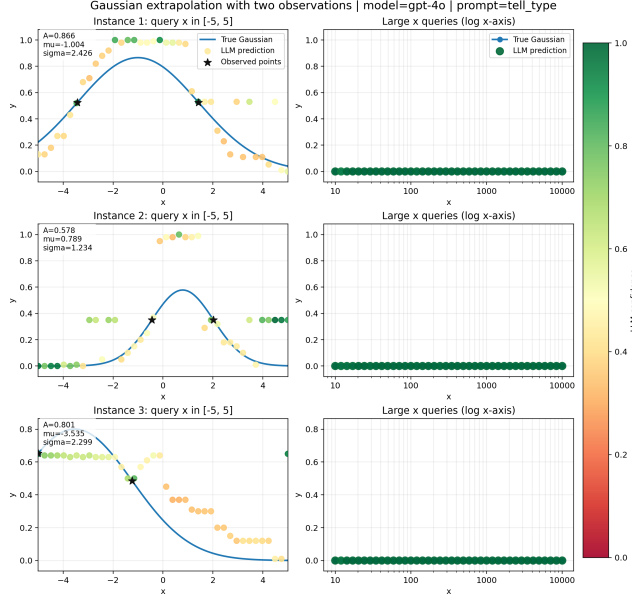


Figure B.3: Pointwise extrapolation to large input values for Gaussian functions. Each row shows one randomly sampled Gaussian instance with two observed points inside $[-5, 5]$. The left panel shows 40 pointwise queries in $[-5, 5]$, and the right panel shows 40 pointwise queries logarithmically spaced in $[10, 10^4]$. Blue curves denote the true function, black stars denote observed points, and colored markers denote LLM predictions. Marker color indicates model confidence, from red (low) to green (high).

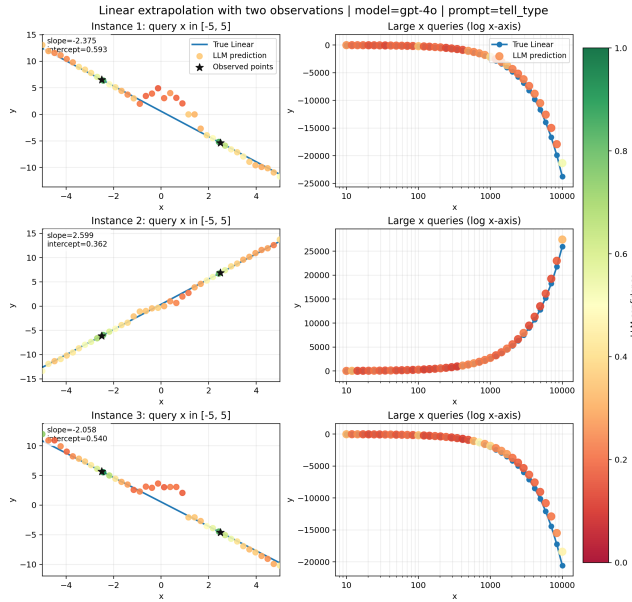


Figure B.4: Pointwise extrapolation to large input values for linear functions. The setup is identical to Figure B.3, except that the underlying functions are linear. Unlike Gaussian functions, the true linear functions continue to grow or decrease outside the observed range, making this a contrasting case for large-scale extrapolation.

We considered two one-dimensional function families: Gaussian functions and linear functions. For each family, we sampled three random function instances using the same function generator as in the main experiments. For each instance, the model was given exactly two observed points sampled inside the standard input range $[-5, 5]$. We then queried the model pointwise at two sets of locations: 40 evenly spaced points in $[-5, 5]$, and 40 logarithmically spaced points in $[10, 10^4]$. Each query was issued as a separate prompt, so the model predicted one scalar output value at a time.

The prompt informed the model of the function family and provided the two observed examples. For example, for a Gaussian function, the prompt stated that the underlying function was Gaussian and then asked the model to predict the value at a single query input. This setup isolates pointwise extrapolation behavior while keeping the observation set fixed.

For Gaussian functions, the true function values decay rapidly toward zero outside the range containing the peak. Therefore, a model that correctly extrapolates the Gaussian form should predict values close to zero for large positive inputs. For linear functions, in contrast, the true function continues to grow or decrease linearly outside the observed range. This provides a useful comparison: Gaussian extrapolation requires recognizing saturation or decay, whereas linear extrapolation requires continuing a simple global trend.

Figure B.3 shows the Gaussian results. Each row corresponds to one sampled Gaussian instance. The left panel shows predictions inside the familiar range $[-5, 5]$, while the right panel shows predictions for large query values on a logarithmic x -axis. The blue curve shows the true function, black stars indicate the two observed points, and colored markers show the LLM predictions. Marker color denotes the model’s confidence, with red indicating low confidence and green indicating high confidence. Across all three instances, the model predicts values close to zero for large positive inputs, consistent with the rapid decay of the Gaussian function.

Figure B.4 shows the corresponding results for linear functions. In contrast to the Gaussian case, the correct extrapolation behavior for a linear function is to continue the global linear trend beyond the observed range. The model’s predictions in the large- x regime remain broadly consistent with this behavior, suggesting that it does not merely collapse to a generic default response for unfamiliar numerical inputs.

Overall, this diagnostic suggests that the model’s pointwise predictions are not restricted to the familiar numerical range $[-5, 5]$. Even when queried at much larger values such as 10 – 10^4 , the model often produces estimates that remain consistent with the stated function family. In particular, the Gaussian case tests whether the model extrapolates the bell-shaped function toward zero at large x , rather than continuing a local trend observed from the two in-range points. The linear case serves as a contrasting setting where large-magnitude extrapolation is mathematically appropriate.

C Verbalizing Shape Bias Under Unknown Function Prompts

The main experiments show that, even when the prompt does not reveal the function family, LLM numerical predictions can exhibit systematic default structure, such as linear or monotonic completions. We therefore ran an auxiliary verbalization experiment to test whether this numerically expressed bias is also available as an explicit verbal judgment.

For each trial, the model was given the same sparse observed points under an unknown-function framing, without being told the true function family or any structural descriptor. Instead of asking for numerical predictions at query locations, we asked the model to describe the pattern in one short sentence, choose one label from a fixed set

$$\mathcal{Y} = \left\{ \begin{array}{l} \text{linear_increasing, linear_decreasing,} \\ \text{bell_shaped, quadratic_upward,} \\ \text{quadratic_downward, periodic,} \\ \text{unclear} \end{array} \right\}.$$

and report a confidence score between 0 and 1. The prompt therefore tests explicit shape attribution rather than pointwise numerical completion.

Figure C.5 shows representative trials. Black points are the observed in-context examples. The small colored points show the corresponding numerical pointwise predictions from the unknown-function condition, included to make the contrast visible. Text inside each panel reports the model’s

verbal trend label and confidence. Green frames mark cases where the verbal label agrees with the apparent/generated structure, while red frames mark clear mismatches.

The examples show that explicit verbalization is often more fragile than numerical prediction. In some trials, the numerical predictions form an approximately increasing linear pattern, but the verbal response is unclear with low confidence. In another trial, the visible pattern is increasing, yet the verbal label is `linear_decreasing`. Conversely, there are cases where the model correctly labels an increasing linear trend. Thus, the same model can expose a structured numerical completion while failing to state that structure reliably in language.

This supports the distinction used throughout the paper. A pointwise numerical answer can be produced by local interpolation, pattern continuation, or other implicit computations over the context. By contrast, a verbal shape description requires the model to aggregate the observed points into a global, discrete structural hypothesis. The latter is a more explicit reasoning task and is not guaranteed to match the computation that generated the numerical predictions. In this sense, verbal reasoning is not equivalent to the optimizer-facing surrogate belief elicited through numerical queries.

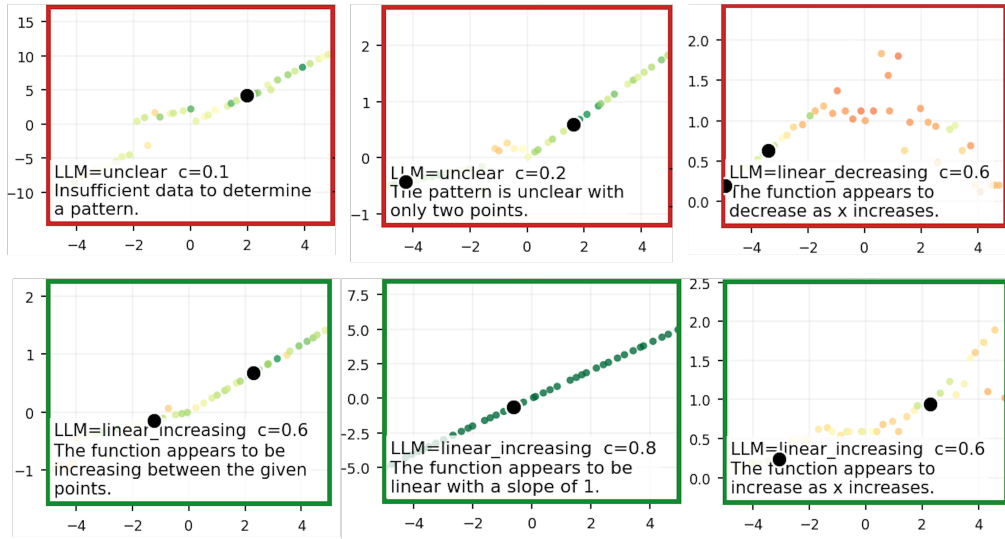


Figure C.5: **Verbalized trend labels under unknown-function prompts.** The model is given sparse observed points from an unknown one-dimensional function and asked to describe the global pattern, choose a trend label, and report confidence. Black points denote observed examples; colored points show the corresponding numerical pointwise predictions from the unknown-function condition. Red frames indicate verbal labels that are inconsistent with the apparent/generated structure, while green frames indicate agreement. The figure illustrates that numerical completions can exhibit linear or monotonic bias even when the model does not reliably verbalize that bias.

D Reference Ambiguity Construction

The reference ambiguity profile $v_{\mathcal{H}}(x; D_t)$ is designed to quantify residual ambiguity induced by the observation set within a controlled function family. It is not intended to be the model’s true posterior variance.

D.1 Generic Monte Carlo construction

Suppose the reference family is parameterized by $\phi \in \Phi$ with function f_{ϕ} . We begin with a broad proposal distribution $p_0(\phi)$ over plausible parameters and generate Monte Carlo proposals

$$\phi^{(m)} \sim p_0(\phi), \quad m = 1, \dots, M. \quad (23)$$

A proposal is retained if it remains consistent with the observations up to tolerance $\varepsilon_{\text{cons}}$, that is, if

$$\max_{(x_i, y_i) \in D_t} |f_{\phi^{(m)}}(x_i) - y_i| \leq \varepsilon_{\text{cons}}. \quad (24)$$

Let $\{\phi^{(1)}, \dots, \phi^{(K)}\}$ denote the retained sample-consistent proposals. These retained samples induce an empirical Monte Carlo reference distribution

$$\hat{q}_D^{\text{MC}} = \frac{1}{K} \sum_{k=1}^K \delta_{\phi^{(k)}}. \quad (25)$$

The reference ambiguity profile is then defined as

$$v_{\mathcal{H}}(x; D_t) = \text{Var}_{\phi \sim \hat{q}_D^{\text{MC}}} [f_{\phi}(x)]. \quad (26)$$

Operationally, this is the empirical variance of the retained function values at each query location:

$$v_{\mathcal{H}}(x; D_t) = \text{Var}(f_{\phi^{(1)}}(x), \dots, f_{\phi^{(K)}}(x)). \quad (27)$$

This is a rejection-style Monte Carlo construction over a controlled reference family. Its purpose is to provide an operational signal of residual sample-consistent ambiguity, not to estimate the model’s true posterior variance or to define a universally correct uncertainty target.

D.2 Why a variance profile?

The sample-consistent family can be summarized in several ways, including pointwise width, envelope area, or posterior-like variance. We use a variance profile for two practical reasons. First, it produces a smooth pointwise ambiguity signal on a fixed query grid, which makes comparison to model-side uncertainty proxies straightforward. Second, the alignment metric in the main text is rank-based, so the precise scale of $v_{\mathcal{H}}(x; D_t)$ is less important than whether it ranks ambiguous regions above less ambiguous ones.

For intuition, a related idealized quantity is the width functional

$$W_{\mathcal{H}}(x; D_t) = \sup_{f, g \in \mathcal{H}(D_t; \mathcal{F})} |f(x) - g(x)|. \quad (28)$$

Both $W_{\mathcal{H}}$ and $v_{\mathcal{H}}$ summarize residual ambiguity in the sample-consistent family, but $v_{\mathcal{H}}$ is especially convenient for Monte Carlo approximation and for direct comparison to model-side uncertainty proxies.

D.3 Families used in the main alignment plots

In the main uncertainty-alignment experiments we use controlled Gaussian and logistic families. These are useful because sparse observations leave visually interpretable regions of residual ambiguity.

For sparse Gaussian examples, ambiguity is often concentrated around unresolved center and width parameters, so retained sample-consistent functions disagree most strongly near the uncertain peak region. For sparse logistic examples, ambiguity is often concentrated around the transition region, where different retained parameter settings induce different slopes and horizontal shifts.

In both cases, the Monte Carlo construction above yields a pointwise profile $v_{\mathcal{H}}(x; D_t)$ that highlights where sample-consistent hypotheses disagree most strongly. This is the reference signal used in Section 4 when evaluating whether model-side uncertainty ranks ambiguous regions above less ambiguous ones.

E Decoding and Protocol-Comparison Details

E.1 Observed-point formatting

All observed examples are rendered with fixed decimal precision, typically two decimals in the prompts shown in the paper. Query outputs are required to follow the same precision. This design reduces the chance that formatting variability is misread as epistemic uncertainty.

E.2 Parsing rules

For pointwise prompts, a valid completion is a single scalar numeric value. For joint prompts, a valid completion is a list of numeric values in the requested order. If a completion contains extra text,

malformed JSON, or the wrong number of outputs, it is marked invalid for numeric analyses. The same parser is used across prompt styles so that any difference in invalidity rate is itself part of the protocol effect rather than an evaluation artifact.

E.3 Repeated sampling

Repeated sampling serves two roles: estimating $u_{\theta}^{\text{sam}}(x)$ and constructing empirical completion distributions for OT analysis. In the protocol-comparison experiments we use 50 repeated samples per condition at the same prompt and temperature configuration. Joint samples yield one vector-valued curve per repetition; pointwise samples are assembled by querying each location independently under the same condition.

E.4 Why protocol affects uncertainty

The protocol difference is not merely about formatting. In JOINT mode, later locations condition on earlier generated outputs, so instability can propagate even if token-level confidence remains high within a sampled sequence. In POINTWISE mode, each location is asked independently against the same evidence set, which more cleanly exposes location-wise ambiguity.

F Sequential Updating Details

This appendix describes how the sequential evidence conditions in Section 5 are constructed, how the GP control is computed, and which auxiliary statistics are used in the sequence-order analysis.

F.1 Experimental setup

For each sampled 1D function instance f^* , we evaluate the surrogate on a fixed query grid

$$\mathcal{X}_{\text{qry}} = \{x^{(1)}, \dots, x^{(41)}\}, \quad (29)$$

which in the plotted experiment is the 41-point grid over $[-5, 5]$. Sequential evidence is then revealed one observation at a time for T steps. For Figure 5, we use $T = 20$ observations, six 1D function families (gaussian, logistic, exponential, quadratic, linear, and sinusoidal), and 10 random instances per family and condition, yielding 300 total sequential runs, or 60 runs per condition. The plotted LLM results use pointwise querying: each query $x \in \mathcal{X}_{\text{qry}}$ is asked separately and token-level confidence is extracted from the numeric response.

F.2 Candidate pool and evidence-order conditions

For each function instance, we first construct a candidate pool of T observations by evaluating the ground-truth function on a uniform grid over the function domain,

$$\mathcal{C} = \{(x_i, y_i)\}_{i=1}^T, \quad y_i = f^*(x_i). \quad (30)$$

All non-conflict conditions use exactly this same pool. Therefore, any difference between `high_info_first`, `low_info_first`, and `random` is caused by evidence order rather than by observing different points.

The main paper reports five evidence-order conditions:

- `high_info_first`: observations judged most informative are revealed early;
- `low_info_first`: observations judged least informative are revealed early;
- `random`: a random permutation of the same candidate pool;
- `conflict_middle`: a contradictory observation is inserted mid-sequence;
- `conflict_late`: a contradictory observation is inserted near the end.

The summary plots in Figure 5 aggregate 60 runs per condition, as indicated in the figure labels.

F.3 Operationalizing informativeness

To rank candidate observations, we use a fixed-hyperparameter GP reference model and score how much a candidate location would reduce posterior spread on the query grid. Let D denote the current revealed prefix. For a candidate location x , we compute a global uncertainty-reduction term

$$g_{\text{global}}(x; D) = \frac{1}{|\mathcal{X}_{\text{qry}}|} \sum_{x' \in \mathcal{X}_{\text{qry}}} [\sigma_D(x') - \sigma_{D \cup \{(x,0)\}}(x')], \quad (31)$$

and a local term centered at x ,

$$g_{\text{local}}(x; D) = \frac{1}{|\mathcal{X}_{\text{qry}}|} \sum_{x' \in \mathcal{X}_{\text{qry}}} w(x', x) [\sigma_D(x') - \sigma_{D \cup \{(x,0)\}}(x')], \quad (32)$$

where $w(x', x)$ is a Gaussian weight centered at x . The final information score is

$$g(x; D) = 0.35 g_{\text{global}}(x; D) + 0.65 g_{\text{local}}(x; D). \quad (33)$$

We use a pseudo-target value 0 in this ranking step because, under fixed kernel hyperparameters, the posterior variance reduction depends on the candidate location rather than the realized target value.

This score operationalizes informativeness as ambiguity reduction on the evaluation grid: points are informative when they collapse posterior spread broadly or in a locally consequential region. The non-conflict orders are then defined by greedy selection under this score:

- `high_info_first`: greedily reveal the remaining candidate with largest $g(x; D)$;
- `low_info_first`: greedily reveal the remaining candidate with smallest $g(x; D)$;
- `random`: reveal a random permutation of the same candidate pool.

Operationally, `high_info_first` tends to place globally constraining points early, whereas `low_info_first` delays those points and initially reveals more redundant or locally limited support. This is why the GP control improves fastest in `high_info_first`, slowest in `low_info_first`, and why the LLM is forced to revise earlier in the former condition.

F.4 Conflict observations

The conflict conditions are designed to test belief revision rather than coverage alone. They are built from a supportive high-information sequence of length $T - 1$. Let the intended conflict insertion step be

$$t_c = \begin{cases} \lfloor T/2 \rfloor + 1, & \text{conflict_middle,} \\ T - 1, & \text{conflict_late.} \end{cases} \quad (34)$$

Using only the supportive prefix before step t_c , we fit the GP reference model and choose the conflict location at the point where the current surrogate is most confident,

$$x_c = \arg \min_{x \in \mathcal{X}_{\text{qry}}} \sigma_{t_c-1}^{\text{GP}}(x). \quad (35)$$

Equivalently, this is the point where a contradictory observation will be maximally surprising relative to the current prefix.

Let

$$\mu_c = \mu_{t_c-1}^{\text{GP}}(x_c), \quad y_c^* = f^*(x_c), \quad s_{\text{obs}} = \max(\text{Std}(y_{1:t_c-1}), 0.2). \quad (36)$$

The magnitude of the contradiction is then chosen by

$$\Delta_c = \lambda \left(0.8 s_{\text{obs}} + 0.6 \sigma_{t_c-1}^{\text{GP}}(x_c) + 0.7 |\mu_c - y_c^*| + 0.25 \right), \quad (37)$$

where λ is a scale parameter (`noise_scale` in code; $\lambda = 3.0$ in the plotted experiment), together with the lower bound

$$\Delta_{\text{min}} = \max(0.8, 0.35\lambda). \quad (38)$$

Finally, the contradictory target is defined as

$$y_c = \mu_c + \eta \max(\Delta_c, \Delta_{\text{min}}), \quad \eta \in \{-1, +1\}. \quad (39)$$

Thus, the conflict point is not merely a low-information point revealed late; it is an explicitly inconsistent observation chosen to overturn the current surrogate. This is why the conflict conditions produce abrupt accuracy drops rather than the slower revision seen in `low_info_first`.

F.5 Per-step LLM evaluation

At each step t , the current prefix $D_t^{(o)}$ is presented to the LLM and the model is queried on the fixed grid $\mathcal{X}_{\text{qry}}$. Mean token-level confidence is

$$\bar{c}_t = \frac{1}{|\mathcal{X}_{\text{qry}}|} \sum_{x \in \mathcal{X}_{\text{qry}}} c_{\theta}^{\text{tok}}(x; D_t^{(o)}, p, \pi). \quad (40)$$

Predictive fit is evaluated on the same grid using

$$\text{Acc}_t = 1 - \text{NRMSE}_t, \quad (41)$$

where NRMSE_t is normalized by the range of the true function values on the query grid. We also summarize the timing of the confidence minimum by

$$t_{\text{valley}} = \arg \min_{1 \leq t \leq T} \bar{c}_t. \quad (42)$$

F.6 GP control replay

To separate model-specific updating from the geometry of the observation sequence, we replay exactly the same prefixes with a GP control using fixed kernel hyperparameters. This control provides a geometric baseline for how sequential evidence alone should affect fit and uncertainty.

For the GP replay, we compute

$$\text{Acc}_t^{\text{GP}} = 1 - \text{NRMSE}_t^{\text{GP}} \quad (43)$$

on the same query grid. Here $\text{NRMSE}_t^{\text{GP}}$ is normalized by the range of the true function values on $\mathcal{X}_{\text{qry}}$, as in the LLM accuracy calculation.

We also report a GP uncertainty proxy,

$$\bar{c}_t^{\text{GP}} = \text{clip}_{[0,1]} \left(1 - \frac{1}{|\mathcal{X}_{\text{qry}}|} \sum_{x \in \mathcal{X}_{\text{qry}}} \sigma_t^{\text{GP}}(x) \right). \quad (44)$$

In this replay, the GP uses fixed kernel hyperparameters with unit signal variance, fixed length scale, no hyperparameter optimization, and no target standardization. Thus $\sigma_t^{\text{GP}}(x)$ is interpreted on the fixed unit-prior GP scale, rather than as an uncertainty measured in the raw target-function units. The clipping only ensures that the displayed proxy lies on the same bounded scale as confidence; it is not used to fit or update the GP.

The dashed curves in Figure 5 correspond to this replay. If a pattern appears in both the LLM and GP curves, it is largely explained by the geometry of the evidence order itself. If the LLM exhibits an additional dip, lag, or mismatch between confidence and accuracy beyond the GP baseline, that extra effect reflects model-side belief updating rather than feasible-set geometry alone.

F.7 Auxiliary stepwise statistics

In addition to confidence and accuracy trajectories, we record two GP-based stepwise reference quantities. First, the surprise of the newly revealed observation under the previous GP belief,

$$\text{Conflict}_t = -\log p_{\text{ref}}(y_t | x_t, D_{t-1}), \quad (45)$$

which is large when the new point is inconsistent with the previously supported trend. Second, we record a GP-based information-gain statistic,

$$\text{InfoGain}_t = \frac{1}{|\mathcal{X}_{\text{qry}}|} \sum_{x \in \mathcal{X}_{\text{qry}}} [L_t(x) - L_{t-1}(x)], \quad (46)$$

where

$$L_t(x) = \log \max_y p_t^{\text{ref}}(y | x). \quad (47)$$

These quantities are used only for analysis and interpretation; the main figure focuses on confidence and accuracy trajectories.

F.8 Trajectory classes

For the supplementary order-comparison panels, we also summarize three coarse trajectory classes: monotone-up (M), U-shape (U), and inverted-U-then-up (I). These labels are descriptive only. They are used to summarize the prevalence of early confidence dips and recoveries across repeated runs, but they are not used as optimization objectives, supervision signals, or training labels.

G Bayesian-Optimization Setup Details

This appendix gives the exact experimental settings for the downstream BO studies in Section 6. The purpose of these experiments is not broad benchmarking, but to test whether protocol-induced differences in surrogate belief lead to different acquisition decisions and optimization outcomes.

G.1 Branin consequence study

Methods. We compare five methods: GP, Random, LLM (Pointwise Neutral), LLM (Pointwise Underdetermination-aware), and LLM (Joint Neutral). In all LLM conditions, the model is not told the underlying function family; the target is always presented as an unknown function. The LLM used in this section is GPT-4o.

Representative LLM prompts. The Branin and shifted-Branin experiments use the same prompt format; only the observed coordinates and values differ from run to run. For readability, we show compact templates rather than full run-specific prompts.

POINTWISE NEUTRAL.

You are a function approximator. Given examples of $((x_1, x_2), y)$ pairs from an unknown 2D function, predict the y value for a new (x_1, x_2) input.

Rules:

1. Output only the numeric y value.
2. Use the same precision as the examples.
3. Do not include explanation or extra text.

Here are example points from an unknown 2D function:

[1] $x = (x_1^{(1)}, x_2^{(1)})$ $y = y^{(1)}$
[2] $x = (x_1^{(2)}, x_2^{(2)})$ $y = y^{(2)}$
...

Given the pattern, predict the y value for:
 $x = (x_1^{\text{query}}, x_2^{\text{query}})$, $y =$

POINTWISE Underdetermination-aware.

Here are example points from an unknown 2D function.

WARNING: This problem is strongly underdetermined. The observed points do not uniquely determine the underlying surface.

Multiple substantially different surfaces, with different slopes, curvature, and local extrema, can fit these same observations equally well.

Do not assume a single confident interpretation from sparse evidence.

Treat this as a high-uncertainty inference task and make a cautious prediction that reflects ambiguity rather than overconfident extrapolation.

[1] $x = (x_1^{(1)}, x_2^{(1)})$ $y = y^{(1)}$
[2] $x = (x_1^{(2)}, x_2^{(2)})$ $y = y^{(2)}$
...

Given this uncertainty, predict the y value for:
 $x = (x_1^{\text{query}}, x_2^{\text{query}})$, $y =$

JOINT NEUTRAL.

You are a function approximator. Given examples of $((x_1, x_2), y)$ pairs from an unknown 2D function, predict the y values for a list of new (x_1, x_2) inputs. Return strict JSON only. Output must contain exactly one key “y” with a list of numeric values in query order.

Here are example points from an unknown 2D function:

[1] $x = (x_1^{(1)}, x_2^{(1)})$ $y = y^{(1)}$
 [2] $x = (x_1^{(2)}, x_2^{(2)})$ $y = y^{(2)}$
 ...

Predict y for each query in order.

Query list:

[1] $x = (u_1^{(1)}, u_2^{(1)})$
 [2] $x = (u_1^{(2)}, u_2^{(2)})$
 ...

Return: $\{“y”: [y1, \dots, yk]\}$

Acquisition rule. At each BO step t , each method scores candidates and selects the top-ranked points. For the GP baseline, we use the standard UCB score

$$a_t^{\text{GP}}(x) = \mu_t^{\text{GP}}(x) + \beta \sigma_t^{\text{GP}}(x),$$

where $\sigma_t^{\text{GP}}(x)$ is the GP posterior predictive standard deviation and $\beta = 3.0$.

For the LLM conditions, we use a UCB-style ranking score

$$a_t^{\text{LLM}}(x) = \mu_t^{\text{LLM}}(x) + \lambda n_t(x),$$

where $\mu_t^{\text{LLM}}(x)$ is the numeric prediction and $n_t(x)$ is the mean token negative log-probability of the numeric output. We treat $n_t(x)$ as a dimensionless exploration feature, not as an objective-scale posterior standard deviation. The coefficient λ is fixed across LLM conditions within the task, so the comparison isolates prompt/protocol effects under the same decision rule.

Budget and candidate sets. Each trial begins with 5 initial observations and then runs for 10 BO steps. At every step, a fresh candidate pool of 80 points is sampled uniformly from the search domain, and the method selects the top 3 points under the acquisition score. These selected points are evaluated and added to the observation set. All methods are evaluated on the same initial observations and the same per-step candidate pools within each trial.

Objectives. We consider two related 2D objectives on the domain $[-5, 10] \times [0, 15]$.

The first is the canonical Branin function, a standard benchmark for Bayesian optimization.

The second is a shifted Branin variant constructed to preserve substantial local resemblance to canonical Branin while moving the optimum to a different location. The purpose of this design is to test whether the LLM optimizer relies on a strong prior over the familiar Branin landscape: if so, such a prior should help on canonical Branin but become misleading once the optimum is displaced.

Construction of modified Branin. The shifted-Branin objective is constructed by evaluating the canonical Branin function after smooth monotone warps of the two input coordinates. Let

$$u = \frac{x_1 + 5}{15}, \quad v = \frac{x_2}{15},$$

so that $(u, v) \in [0, 1]^2$. The warped coordinates are

$$\tilde{x}_1 = -5 + 15 \phi_1(u), \quad \tilde{x}_2 = 15 \phi_2(v),$$

where each warp is an endpoint-normalized logistic transform

$$\phi(z; k, b) = \frac{\sigma(k(z - b)) - \sigma(-kb)}{\sigma(k(1 - b)) - \sigma(-kb)}, \quad \sigma(t) = \frac{1}{1 + e^{-t}}.$$

In the implementation, the two axes use different parameters:

$$\phi_1(u) = \phi(u; 7.5, 0.33), \quad \phi_2(v) = \phi(v; 5.5, 0.62).$$

The shifted objective is then

$$f_{\text{shift}}(x_1, x_2) = f_{\text{Branin}}(\tilde{x}_1, \tilde{x}_2).$$

Thus the function preserves the overall Branin topology, but the basin locations and shapes are displaced by anisotropic nonlinear warping.

Camouflaged initialization for modified Branin. In the shifted-Branin condition, the objective is constructed as a Branin-like landscape with an additional hidden well, so that its global optimum is displaced from the canonical Branin minima. Writing the canonical Branin objective as $f_B(x)$ and the shifted objective as $f_{\text{shift}}(x)$, the initialization set is not sampled arbitrarily. Instead, candidate initial points are drawn uniformly from the domain and retained only if

$$|f_{\text{shift}}(x) - f_B(x)| < \tau,$$

with a strict threshold $\tau = 2$ and a relaxed threshold $\tau = 2.5$ if too few points satisfy the stricter condition. Thus, each initial observation is individually chosen so that the shifted objective remains close in value to canonical Branin.

Among the retained candidates, the final set of initial points is chosen to minimize a camouflage score that favors: (i) small average value gap between shifted and canonical Branin, (ii) low pairwise ranking disagreement between the two objectives on the selected points, and (iii) low proximity to the hidden well, while also encouraging spatial spread. Formally, for an initialization set S ,

$$\text{Score}(S) = \frac{1}{|S|} \sum_{x \in S} |f_{\text{shift}}(x) - f_B(x)| + 2 \text{RankMismatch}(S) + \text{WellProximity}(S) - 0.3 \text{Spread}(S),$$

and the set with the lowest score is used. This creates a deliberately camouflaged prefix of evidence: the early observations are broadly consistent with the standard Branin landscape even though the true optimum lies elsewhere. The goal is to test whether the optimizer becomes anchored to the familiar prior and is therefore drawn toward a decoy basin.

Metrics. Our main metric is best-so-far simple regret after each BO step. We write the objective in reward form, so larger values are better and the optimal value is normalized to 0. Regret is therefore

$$r_t = y^* - \max_{s \leq t} y_s,$$

with $y^* = 0$. We also record whether the selected batch contains a top-10% candidate in the current pool and compute the Spearman rank correlation between surrogate mean predictions and true candidate values, although these auxiliary diagnostics are not the main focus of Section 6.

Repeated trials. Each condition is repeated over 10 trials. Reported curves show the evolution of best-so-far regret across BO steps, and the figure also compares the distribution of final regret across trials.

G.2 XGBoost HPO

The realistic HPO experiment is a fixed-library Bayesian optimization task over XGBoost hyperparameters. The main-text panel reports the California Housing regression task only. The hidden objective is XGBoost validation performance on a fixed train/validation split. For regression, the optimizer stores negative validation RMSE, so larger values are better.

Search space and library. The search space contains ten XGBoost hyperparameters: learning rate, maximum depth, minimum child weight, row subsampling, column subsampling, L2 regularization, L1 regularization, split penalty, number of boosting rounds, and histogram bin count. The ranges are deliberately wide, including underfitting and over-regularized regions, so that random search is not trivially competitive. A Sobol library of 128 configurations is precomputed and reused by all methods. BO seeds change the initial observations and search trajectory, not the underlying objective.

Budget and candidate sets. Each run starts from 3 initial observations and then performs 20 one-point BO steps. The per-step candidate pool is shared across methods within a seed. In the plotted run, the candidate-pool size is 128, which means that after removing already observed configurations, each method scores essentially all remaining configurations in the fixed library.

Methods and acquisition. The plotted methods are Random, SKOpt-style GP+EI, Optuna-style TPE, SMAC3-style random forest+EI, LLM Pointwise Neutral, and LLM Joint Neutral. The LLM model is Qwen3.5-397B-A17B. The saved run also contains a pointwise warning condition, but the main panel focuses on the neutral protocol comparison. The classical surrogate baselines use their standard expected-improvement implementations. For LLM methods, the predicted mean is the model’s predicted validation objective, and the mean negative log-probability of the numeric output tokens is used as a heuristic exploration feature in an EI-style ranking score. This LLM score is used only to rank candidates within the same BO step; it should not be interpreted as calibrated expected improvement under a Gaussian posterior.

LLM-visible information. The LLM sees the model family (XGBoost), task type, sample count, feature count, observed hyperparameter configurations with their objective values, and either one candidate (POINTWISE) or a list of candidates (JOINT). It does not see the dataset name, OpenML identifier, or any text identifying California Housing.

Representative LLM prompts. For readability, we show compact templates rather than full candidate lists.

POINTWISE NEUTRAL.

You are optimizing XGBoost hyperparameters from past trials.
Predict the validation objective value for one candidate configuration.
Objective: RMSE (lower is better).

Model family: XGBoost gradient-boosted trees
Task type: regression
Samples: [number of training examples]
Features: [number of input features]

Observed trials:
[1] [hyperparameter configuration 1] → objective (RMSE) = [observed value 1]
[2] [hyperparameter configuration 2] → objective (RMSE) = [observed value 2]
...

Candidate:
candidate hyperparameter configuration

Return only one numeric objective value.

JOINT NEUTRAL.

You are optimizing XGBoost hyperparameters from past trials.
Predict validation objective values for multiple candidate configurations.
Objective: RMSE (lower is better).

Model family: XGBoost gradient-boosted trees
Task type: regression
Samples: [number of training examples]
Features: [number of input features]

Observed trials:
[1] [hyperparameter configuration 1] → objective (RMSE) = [observed value 1]
[2] [hyperparameter configuration 2] → objective (RMSE) = [observed value 2]
...

Candidates in order:
[1] [candidate configuration 1]

[2] [candidate configuration 2]

...

Return strict JSON only: {"scores": [s1, s2, s3, ...]}

Outcome summary. Figure 7 reports raw best-so-far simple regret relative to the best configuration in the fixed library. Mean final raw regret over ten seeds is 0.0043 for SMAC3, 0.0087 for GP+EI, 0.0107 for LLM Pointwise Neutral, 0.0108 for LLM Joint Neutral, 0.0141 for Random, and 0.0217 for TPE. Thus the LLM pointwise surrogate is competitive with standard BO baselines on this task, but SMAC3 is more reliable by the end of the budget.

G.3 Battery cathode-design fixed-pool BO

The battery experiment is a fixed-pool BO task over PyBaMM DFN simulations of a redesigned Chen2020 full cell. We optimize two active cathode-design variables,

$$L_{\text{pos}} \in [4.0 \times 10^{-5}, 1.5 \times 10^{-4}] \text{ m}, \quad \epsilon_{\text{pos}} \in [0.22, 0.45],$$

Objective. For each candidate design, PyBaMM evaluates the delivered volumetric energy density at both low-rate and high-rate discharge, denoted by ED(0.5C) and ED(3C), in Wh/L. We normalize these two quantities by rate-specific reference values,

$$Q_{\text{low}} = 943.05 \text{ Wh/L}, \quad Q_{\text{high}} = 509.35 \text{ Wh/L},$$

defined as the 90th percentiles of successful pilot-sweep energy densities at 0.5C and 3C. The normalized components are

$$s_{\text{low}} = \frac{\text{ED}(0.5\text{C})}{Q_{\text{low}}}, \quad s_{\text{high}} = \frac{\text{ED}(3\text{C})}{Q_{\text{high}}}.$$

The scalar score optimized by BO is

$$\text{score} = \frac{2s_{\text{high}}s_{\text{low}}}{s_{\text{high}} + s_{\text{low}}} + 0.25 \mathbb{I}[s_{\text{high}} \geq 1.0 \text{ and } s_{\text{low}} \geq 0.9].$$

Higher score is better. The harmonic-mean term rewards designs that perform well at both rates and penalizes one-sided solutions; the bonus rewards candidates that simultaneously clear the high-rate and low-rate thresholds.

Pool and budget. We evaluate BO on a fixed discrete candidate pool of 200 designs sampled from a larger PyBaMM sweep. The pool is deliberately imbalanced to mimic sparse high-performing regions: 5 high-score designs have scores at least 0.95, 25 intermediate designs have scores between 0.50 and 0.90, and 170 low-score decoys have scores at most 0.20. The maximum score in the pool is 1.2123. Therefore, only $5/200 = 2.5\%$ of candidates exceed the 0.95 high-score threshold, and only $2/200 = 1.0\%$ achieve a score of at least 1.0.

Each run starts from 3 initial observations and performs up to 20 sequential one-point BO acquisitions over the remaining candidates. A run is terminated early once the global maximum in the pool is observed. For plotting only, terminated trajectories are padded with their final best-so-far value so that all runs have the same length.

Methods and acquisition. We compare Random, GP+EI, TPE, SMAC3, LLM Pointwise Neutral, and LLM Pointwise Warning. The LLM model is GPT-5.4. The LLM sees the active physical variables, bounds, the fixed value of R_{pos} , observed designs with score and rate-wise components, and one candidate at a time. As in the HPO experiment, LLM uncertainty is derived from token-level uncertainty, and all non-random methods use expected improvement.

Representative LLM prompts. We show concise templates rather than full run-specific candidate logs.

POINTWISE NEUTRAL.

You are optimizing a battery cathode design with two active variables: L_{pos} and ϵ_{pos} . Maximize energy density under both low-rate (0.5C) and high-rate (3C) conditions. Prefer high and balanced performance across both rates.

Score:

$$s_{\text{low}} = \text{ED}(0.5\text{C})/Q_{\text{low}}$$

$$s_{\text{high}} = \text{ED}(3\text{C})/Q_{\text{high}}$$

$$\text{score} = 2s_{\text{high}}s_{\text{low}}/(s_{\text{high}} + s_{\text{low}}) + 0.25\mathbb{I}[s_{\text{high}} \geq 1.0 \text{ and } s_{\text{low}} \geq 0.9].$$

Higher score is strictly better.

Observed trials so far:

[1] [design 1] $\rightarrow$ score = [value 1] (s_{low} = [low-rate component], s_{high} = [high-rate component])

...

Candidate to score:

candidate design

Predict the expected `score` for this candidate using the formula above.

Return only one numeric value.

POINTWISE Warning.

The warning condition uses the same neutral prompt with the following additional block:

IMPORTANT PHYSICS WARNING

PAY CLOSE ATTENTION TO THIS. DO NOT IGNORE THE RATE-DEPENDENT PHYSICS:

- Low-rate energy density improves with lower inactive content, such as lower porosity.

- At high rate, transport pathways and porosity become more important.

Use this warning aggressively when judging candidates. A design that looks good at only one rate is not enough; the score rewards high, balanced performance across both low-rate and high-rate discharge.

Outcome summary. Figure 8 reports best-so-far regret relative to the pool optimum. The Warning prompt gives the strongest final performance: its mean final regret is 0.0500, it reaches the treasure threshold 0.95 in all 10/10 seeds, and it finds the exact global optimum in 8/10 seeds. SMAC3 is the closest non-LLM competitor, with mean final regret 0.0574, treasure-threshold hit rate 9/10, and global-optimum hit rate 6/10.

The Neutral prompt also improves substantially over other non-LLM search, reaching the treasure threshold in 9/10 seeds and the global optimum in 5/10 seeds, with mean final regret 0.1310. The remaining baselines have larger mean final regret: 0.2284 for TPE, 0.2696 for Random, and 0.2718 for GP+EI. Although TPE and GP+EI occasionally find the optimum, their high variance leads to worse mean final regret. Overall, the result shows that the pointwise LLM surrogate can be competitive on this sparse fixed-pool battery-design problem, and that adding the physics warning yields the most reliable performance.

H Licenses and Existing Assets

We use existing language models, software packages, datasets, and benchmark objectives only for evaluation. OpenAI models are accessed via the OpenAI API and are subject to OpenAI’s Terms of Use. Qwen models are used under their respective open-source licenses (e.g., Apache License 2.0 for certain releases), and Llama models are used under the Meta Llama Community License Agreement.

We use XGBoost, Optuna, SMAC3, PyBaMM, NumPy, SciPy, scikit-learn, and Matplotlib under permissive open-source licenses (e.g., BSD, MIT, Apache 2.0). The California Housing dataset is accessed through scikit-learn as a standard public regression benchmark. The Branin objective is a standard synthetic benchmark, and the shifted Branin and battery-design candidate pool are generated as described in Appendix G.

We do not redistribute third-party model weights, datasets, or software packages. All assets are used in accordance with their respective licenses or terms of use.

I Key implementation constants

Table I.2 summarizes the main implementation constants needed to reproduce the ambiguity profiles, metric scales, GP controls, and LLM-based acquisition scores. These constants were fixed before evaluation and were not tuned separately for individual prompts, protocols, or models. For LLM-based BO, token-level negative log-probability is used as a heuristic exploration feature rather than as a calibrated objective-scale posterior standard deviation.

Table I.2: Key implementation constants.

Component	Quantity	Value / setting
Reference ambiguity profiles	Query grid	$[-5, 5]$ with 41 query points; observed anchors handled separately
FRS	Scale parameter	$\tau_f = \text{IQR}(y)^2$; aggregate protocol run used $\tau_f = 0.64$
Sequential GP control	GP kernel and nugget	Constant kernel amplitude 1.0 times RBF length-scale 1.0; nugget 10^{-6}
Sequential information score	Local weighting	Global/local weights 0.35/0.65; local Gaussian bandwidth 1.5
Conflict construction	Conflict strength	$\lambda = 3.0$ and $\Delta_{\min} = \max(0.8, 0.35\lambda)$
LLM token uncertainty	Aggregation	Mean negative log-probability over numeric-output tokens
Branin BO	LLM acquisition score	$\mu_t^{\text{LLM}}(x) + 3.0 n_t(x)$
HPO and battery BO	LLM uncertainty feature	Mean token NLL used as heuristic uncertainty feature, floored at 10^{-6}

Here, $\text{IQR}(y)$ denotes the interquartile range of the target values used to set the FRS scale for the corresponding aggregate evaluation, and $n_t(x)$ denotes the mean token negative log-probability of the numeric output.